20.01 The purposes of an operating system (OS)

Chapter 8 (Section 8.02) contained a categorised summary of the various activities that an operating system engages in. This chapter discusses some of them in more detail.

We can begin by considering a few aspects relating to the use of an OS.

1. A computer system needs a program that begins to run when the system is first switched on. At this stage, the operating system programs are stored on disk so there is no operating system. However, the computer has stored in ROM a basic input/output system (BIOS) which starts a bootstrap program. It is this bootstrap program that loads the operating system into memory and sets it running.
2. An operating system can provide facilities to have more than one program stored in memory. Only one program can access the CPU at any given time but others are ready when the opportunity arises, which is described as multi-programming and will happen for one single user. Some systems are designed to have many users simultaneously logged in, which is described as a time-sharing system.
3. The purposes of an operating system can usefully be considered from two viewpoints: an internal viewpoint and an external viewpoint. The internal viewpoint concerns how the activities of the operating system are organised to best use the resources available. The external viewpoint concerns the facilities made available for the user of the system.
4. The major resources associated with the internal viewpoint are the CPU, the memory and the I/O system.

Operating system structure

An operating system is structured to provide a platform for both resource management and the provision of facilities for users. The logical structure of the operating system provides two modes of operation. User mode is the one available for the user or an application program. The alternative has a number of different names but the most often used is 'kernel mode'. The difference between the two is that kernel mode has sole access to part of the memory and to certain system functions that user mode cannot access.

The parts of the OS that provide the two modes of operation are separated. The kernel runs all of the time. The remainder of the OS runs in user mode so individual parts are only accessed when needed. One possibility then is to use a layered structure as illustrated in Figure 20.01. In this model, application programs or utility programs could make system calls to the kernel. However, to work properly each higher layer needs to be fully serviced by a lower layer (as in a network protocol stack).

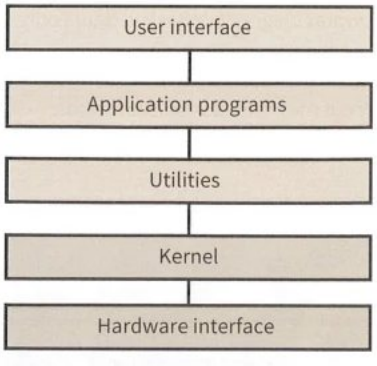


Figure 20.01 Layered structure for an operating system

A layered structure for an operating system is hard to achieve in practice. A more flexible approach uses a modular structure, illustrated in Figure 20.02. The structure works by the kernel calling on the individual services when required. It could be associated with a micro-kernel structure where the functionality in the kernel is reduced to the absolute minimum.

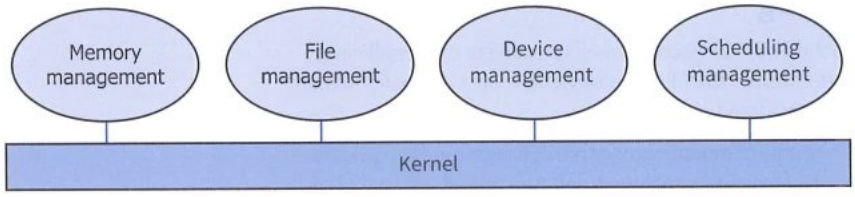


Figure 20.02 Modular structure for an operating system

20.02 The input/output (I/O) system

The I/O system does not just relate to input and output that directly involves a computer user. It also includes input and output to storage devices while a program is running. Figure 20.03 is a schematic diagram that illustrates the structure of the I/O system.

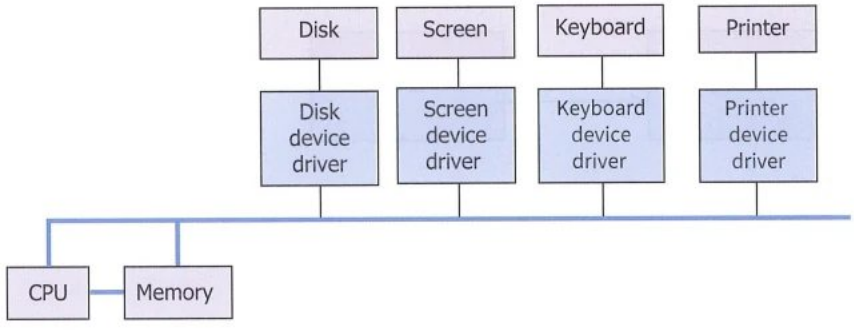


Figure 20.03 Main components associated with the I/O system

The bus structure in Figure 20.03 shows that there can be an option for the transfer of data between an I/O device and memory. The operating system can ensure that I/O passes via the CPU but for large quantities of data the operating system can ensure direct transfer between memory and an I/O device.

To understand the issues associated with I/O management, some discussion of timescales is required. It must be understood that one second is a very long time for a computer system. A CPU typically operates at GHz frequencies. One second sees more than one trillion clock cycles. Some typical speeds for I/O are given in Table 20.01.

Device	Data rate	Time for transfer of 1 byte
Keyboard	10 Bps	0.1 s
Screen	50 MBps	2×10^{-8} s
Disk	5 MBps	2×10^{-7} s

Table 20.01 Typical rates and times for data transfer

The slow speed of I/O compared to a typical CPU clock cycle shows that management of CPU usage is vital to ensure that the CPU does not remain idle while I/O is taking place. This is discussed in the next section.

20.03 Process scheduling

Resource management relating to the CPU mainly concerns scheduling to ensure efficient usage. The methods discussed here consider the CPU as a single unit; specific issues relating to a multiprocessor system are not considered.

Programs that are available to be run on a computer system are initially stored on disk. A user could submit a program as a 'job' which would include the program and some instructions about how it should be run. Figure 20.04 shows an overview of the components involved when a program is run.

A long-term or **high-level scheduler** program controls the selection of a program stored on disk to be moved into main memory. Occasionally a program has to be taken back to disk due to the memory getting overcrowded. This is controlled by a medium-term scheduler. When the program is installed in memory, a short-term or **low-level scheduler** controls when it has access to the CPU.

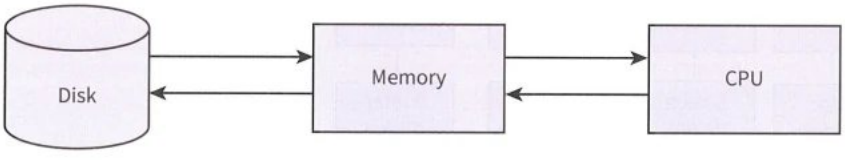


Figure 20.04 Components involved in running a program

KEY TERMS

High-level scheduler: makes decisions about which program stored on disk should be moved into memory

Low-level scheduler: makes decisions about which process stored in memory should have access to the CPU

Process states

In Chapter 8 (Section 8.02), it was stated that a **process** can be defined as 'a program being executed'. This definition can be improved by including the state when the program first arrives in memory. At this stage a **process control block (PCB)** can be created in memory ready to receive data when the process is executed. Once in memory the state of the process can change.

The transitions between the states shown in Figure 20.05 can be described as follows.

- A new process arrives in memory and a PCB is created; it changes to the ready state.
- A process in the ready state is given access to the CPU by the dispatcher; it changes to the running state.
- A process in the running state is halted by an interrupt; it returns to the ready state.
- A process in the running state cannot progress until some event has occurred (I/O perhaps); it changes to the waiting state (sometimes called the 'suspended' or 'blocked' state).

- A process in the waiting state is notified that an event is completed; it returns to the ready state.
- A process in the running state completes execution; it changes to the terminated state.

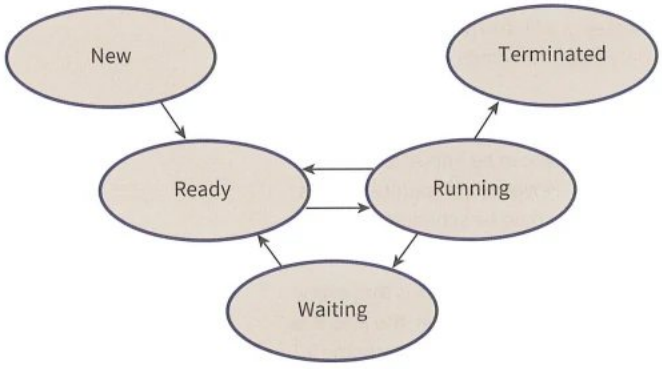


Figure 20.05 The five states defined for a process being executed

It is possible for a process to be separated into different parts for execution. The separate parts are called **threads**. If this has happened, each thread is handled as an individual process.

KEY TERMS

Process: a program in memory that has an associated process control block

Process control block (PCB): a complex data structure containing all data relevant to the running of a process

Thread: part of a program which is handled as an individual process when being executed

Interrupts

Some interrupts are caused by errors that prematurely terminate a running process. Otherwise there are two reasons for interrupts.

- Processes consist of alternating periods of CPU usage and I/O usage. I/O takes far too long for the CPU to remain idle waiting for it to complete. The interrupt mechanism is used when a process in the running state makes a system call requiring an I/O operation and has to change to the waiting state.
- The scheduler decides to halt the process for one of several reasons as discussed later under the heading 'Scheduling algorithms'.

Whatever the reason for an interrupt, the OS kernel must invoke an interrupt-handling routine. This may have to decide on the priority of an interrupt. One required action is that the current values stored in registers must be recorded in the process control block. This allows the process to continue execution when it eventually returns to the running state.

Discussion Point:

What would happen if an interrupt was received while the interrupt-handling routine was being executed by the CPU? Does this require a priority being set for each interrupt?

Scheduling algorithms

Although the long-term or high-level scheduler will have decisions to make when choosing which program should be loaded into memory, we concentrate here on the options for the short-term or low-level scheduler.

A scheduling algorithm can be preemptive or non-preemptive. A preemptive algorithm can halt a process that would otherwise continue running undisturbed. If an algorithm is preemptive it may involve prioritising processes.

The simplest possible algorithm is first come first served (FCFS). This is a non-preemptive algorithm and can be implemented by placing the processes in a first-in first-out (FIFO) queue. The algorithm will be very inefficient if it is the only algorithm employed but it can be used as part of a more complex algorithm.

A round-robin algorithm allocates a time slice to each process and is therefore preemptive, because a process will be halted when its time slice has run out. It can be implemented as a FIFO queue. It normally does not involve prioritising processes. However, if separate queues are created for processes of different priorities then each queue could be scheduled using a round-robin algorithm.

A priority-based scheduling algorithm is more complicated. One reason for this is that every time a new process enters the ready queue or when a running process is halted, the priorities for the processes may have to be re-evaluated. The other reason is that whatever scheme is used to judge priority level it will require some computation. Possible criteria are:

- estimated time of process execution
- estimated remaining time for execution
- length of time already spent in the ready queue
- whether the process is I/O bound or CPU bound.

More than one of these criteria might be considered. Clearly, estimating a time for execution may not be easy. Some processes require extensive I/O, for instance printing wage slips for employees. There is very little CPU usage for such a process so it makes sense to allocate it a high priority so that the small amount of CPU usage can take place. The process will then change to the waiting state while the printing takes place.

20.04 Memory management

The term memory management includes a number of aspects.

- The provision of protected memory space for the OS kernel.
- The loading of a program into memory requires defining the memory addresses for the program itself, for associated procedures and for the data required by the program.
- In a multiprogramming system, this might not be straightforward. The storage of processes in main memory can get fragmented in the same way as happens for files stored on a hard disk. There may be a need for the medium-term scheduler to move a process out of main memory to ease the problem.
- Decisions have to be made about how large a part of memory should be allocated to individual processes sharing memory.

Partitions and segments

An early approach to memory management when different processes were loaded into memory simultaneously was to partition the memory. The aim was to load the whole of a process into one partition. This was wasteful of memory if the process size was less than the partition size. An improvement was dynamic partitioning where the partition size was allowed to adjust to match the process size. The rule of one process per partition remained.

An extension of this idea which allowed for larger processes to be handled was **segmentation**. The large process was divided into segments. Each segment was loaded into a dynamic partition in memory.

There were two factors that limited the efficiency of this approach. The first was that the segments were not constrained to be the same size. The second was that the size of process did not allow all of the segments for one process to be in memory at the same time. Segments had to be moved from disk to memory but then back again to disk when a different segment was needed in memory.

These two factors combined to cause fragmentation both of the memory and of the disk storage. This resulted in degradation in the performance of the system.

Paging and virtual memory

The modern approach is to use **paging**. The process is divided into equal-sized pages and memory is divided into frames of the same size. The secondary storage can also be divided into frames.

KEY TERMS

Segmentation: where a large process is divided into segments for loading into memory but the segments are not constrained to be the same size

Paging: where a large process is divided into pages which have to be the same size

It could be possible to load all of the pages into memory at the same time. However, even if this were possible it is usually the case that not all parts of the program are needed at the same time. A large program is likely to have optional routes for the execution.

A special case for the use of paging is when a program is so large that the address space needed for it is larger than the size of the memory. Paging now supports what is known as **virtual memory** management.

KEY TERM

Virtual memory: a paging mechanism that allows a program to use more memory addresses than are available in main memory

Paging requires the CPU to transfer address values to a memory management unit that allocates a corresponding address on a page. An address has to comprise two parts: the page number plus the offset from the start of the page. The memory management unit functions through the use of a page table. Figure 20.06 shows a very simplified system sufficient to illustrate how this works.

The left-hand part of Figure 20.06 shows a program with 48 instructions. These instructions occupy three pages. The three pages occupy the first 48 logical memory addresses. Because only 8 bits are used to store the logical address there are only 16 pages allowed. The logical address stores the page number in the four most significant bits and the page offset in the four least significant bits.

The right-hand part of Figure 20.06 shows the page frames with physical memory addresses. It shows a scenario where the first two program pages have been loaded into page frames. Note that the page frames used do not have to be adjacent. The centre part of Figure 20.06

shows the contents of the page table for this process (there will be a separate page table for each process in memory). This table has the page number functioning as an index. It has a value for the presence flag indicating whether or not the page is in memory. In the version shown here the third entry shows the page frame number. This might instead have recorded the physical address of the first item in the page frame.

TIP

Remember that you will often see numbering starting from 0.

Logical address	Page number	Program
11111111 ⋮ 11110000	15	
⋮	⋮	
11101111 ⋮ 11100000	2	Instr47 ⋮ Instr32
00011111 ⋮ 00010000	1	Instr31 ⋮ Instr16
00011111 ⋮ 00000000	0	Instr15 ⋮ Instr00

Page number	Presence flag	Page frame number
2	0	
1	1	3
0	1	1

Physical address	Program
⋮	
00111111 ⋮ 01100000	Instr31 ⋮ Instr16
00101111 ⋮ 00100000	
00011111 ⋮ 00010000	Instr15 ⋮ Instr00
00001111 ⋮ 00000000	

Figure 20.06 The use of a page table for a paging system

When paging is being used the starting situation is that the set of pages comprising a process are stored on disk. One or more of these pages is loaded into memory when the process is changing to the ready state. When the process is dispatched to the running state, the process starts executing. At some stage, the process will need access to a page that the page table indicates is not in memory. This is called a page fault condition. It is now almost inevitable that, in order to bring in the required page from secondary storage, a page will need to be taken out of memory first. This is when a page replacement algorithm is needed. There are a number of options for this. A simple algorithm would use a first-in first-out method. A more sensible method would be the least-recently-used page but this requires statistics of page use to be recorded.

The system overhead in running virtual memory can be a disadvantage. The worst problem is **disk thrashing**, when part of a process on one page requires another page which is on disk. When that page is loaded it almost immediately requires the original page again.

This can lead to almost never-ending loading and unloading of pages. Algorithms have been developed to guard against this never-ending loading and unloading but the problem can still occur.

KEY TERM

Disk thrashing: when paging is being used and a repetitive state has been reached where loading one page causes a need for another page to be loaded almost immediately but the loading of this new page causes the same immediate need

Extension Question 20.01

Find out information about the virtual memory capacity and page size of a computer system that you use. In what circumstances might these need to be changed?

20.05 Operating system facilities provided for the user

The user interface may be made available as a command line, a graphical display or a voice recognition system. But the function of a user interface is always to allow the user to interact with running programs. When a program involves use of a device, the operating system provides the device driver: the user just expects the device to work. (You might, however, wish to argue that printers do not always quite fit this description.)

The operating system will provide a file system for a user to store data and programs. The user has to choose filenames and organise a directory (folder) structure but the user does not have to organise the physical data storage on a disk. If the user is a programmer, the operating system supports the provision of a programming environment. The operating system allows a program to be created and run without the programmer being familiar with how the processor functions.

When a program is running it can be considered to be a type of user. The operating system provides a set of system calls that provide an interface to the services it offers. For instance, if a program specifies that it needs to read data from a file, the request for the file is converted into a system call that causes the operating system to take charge, find the file and make it available to the program. An extension of this concept is when an operating system provides an application programming interface (API). Each API call fulfils a specific function such as creating a screen icon. The API might use one or more system calls. The API concept aims to provide portability for a program, where a program can run on different operating systems with minimal changes.

20.06 Translation software

Chapter 8 (Section 8.05) provided an overview of how a compiler or an interpreter is used. We will now will consider some details of how a compiler works with a brief reference to the workings of an interpreter. Compiler or interpreter writing is a specialised task carried out by professionals who each will have their own particular methods. So, we will just look at some general principles and illustrations of some of the techniques that are likely to be used.

A compiler can be described as having a 'front end' and a 'back end'. The front-end program performs analysis of the source code and unless errors are found produces an intermediate code that expresses completely the semantics (the meaning) of the source code. The back-end program then takes this intermediate code as input and performs synthesis of object code. This analysis–synthesis model is represented in Figure 20.07.